Investigación

Certificados de Seguridad SSL (Secure Sockets Layer):
¿Qué es SSL, para que sirve y cómo funciona? ¿Cómo se identifica visualmente un navegador que se está utilizando SSL? Realizar una gráfica del funcionamiento. ¿Cuál es la diferencia entre HTTP y HTTPS? ¿Cómo puede obtener un certificado SSL gratuito?

SSL (*Secure Sockets Layer*) es una tecnología que **protege la comunicación entre tu dispositivo y un sitio web** para que nadie más pueda leerla ni modificarla. Su versión moderna se llama TLS.

Para qué sirve

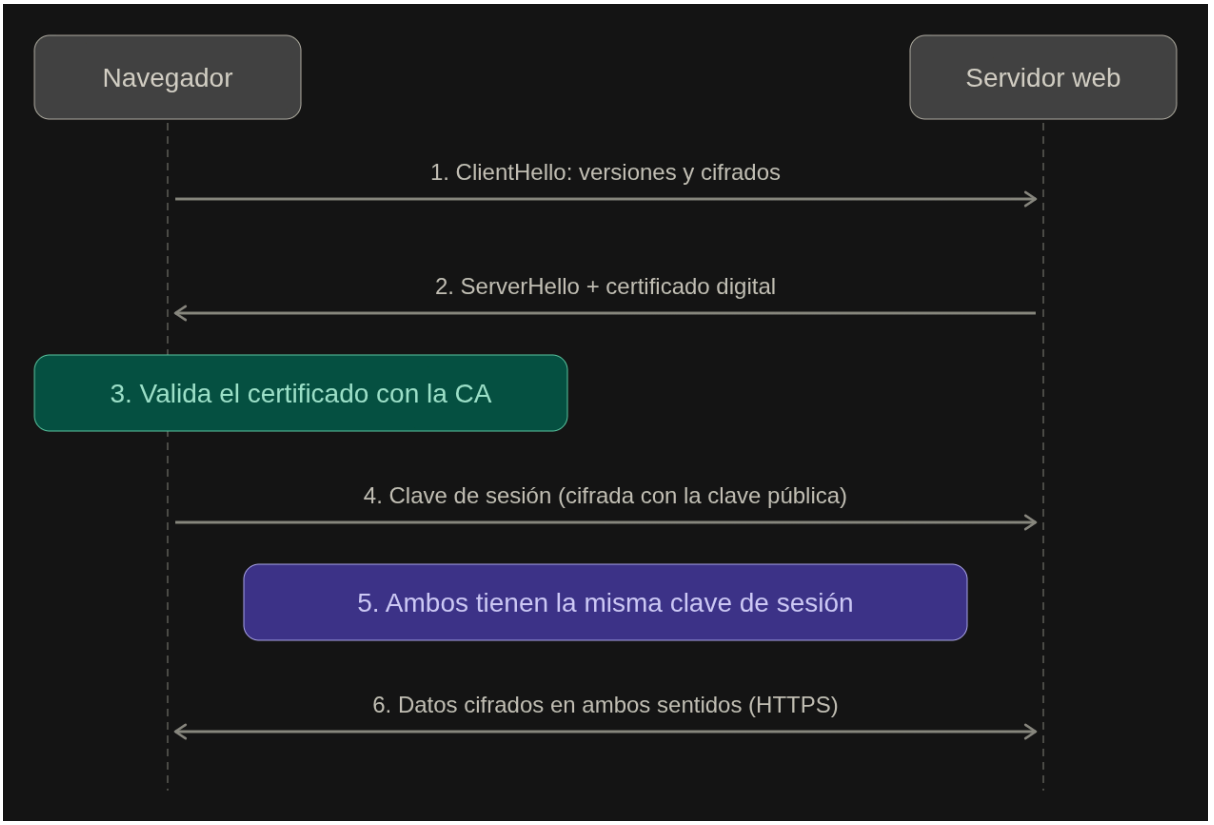
1. **Cifrar la información.** Contraseñas, números de tarjeta y mensajes viajan codificados. Si alguien los intercepta, ve solo caracteres sin sentido.
2. **Verificar la identidad del sitio.** Te asegura que estás hablando con el banco real y no con una copia falsa que se hace pasar por él.
3. **Evitar alteraciones.** Garantiza que nadie cambie el contenido en el camino, por ejemplo para inyectar publicidad o código malicioso.

Cómo funciona

1. **El saludo.** Tu navegador le dice al servidor: “Quiero una conexión segura. Estos son los métodos de cifrado que entiendo”.
2. **El certificado.** El servidor responde con su **certificado digital**. Incluye su clave pública y está firmado por una autoridad certificadora (CA) en la que tu navegador ya confía.
3. **La verificación.** Tu navegador comprueba que el certificado sea válido, que no haya vencido y que corresponda realmente a ese dominio. Si algo falla, te muestra una advertencia.
4. **El acuerdo de una clave secreta.** Ambos usan la clave pública y la privada del servidor para ponerse de acuerdo en una **clave de sesión**.
5. **La comunicación cifrada.** A partir de ahí, todo lo que se envían (páginas, formularios, contraseñas) se cifra y descifra con esa clave de sesión.

Cómo se identifica visualmente que un sitio usa SSL/TLS

- **La URL empieza con `https://`** en lugar de `http://`.
- **Aparece un ícono junto a la dirección.** Antes era un candado cerrado. Chrome lo reemplazó por un ícono de “ajustes” (dos líneas con círculos), pero el sitio sigue siendo seguro. Firefox, Safari y Edge todavía muestran el candado.
- **Al hacer clic en ese ícono** se ve “La conexión es segura” y se puede consultar el certificado: quién lo emitió, a nombre de quién está y cuándo vence.
- **Si algo falla**, el navegador muestra “No seguro” o una pantalla roja de advertencia. Eso ocurre con certificados vencidos, mal configurados o no confiables.



Diferencias entre HTTP y HTTPS

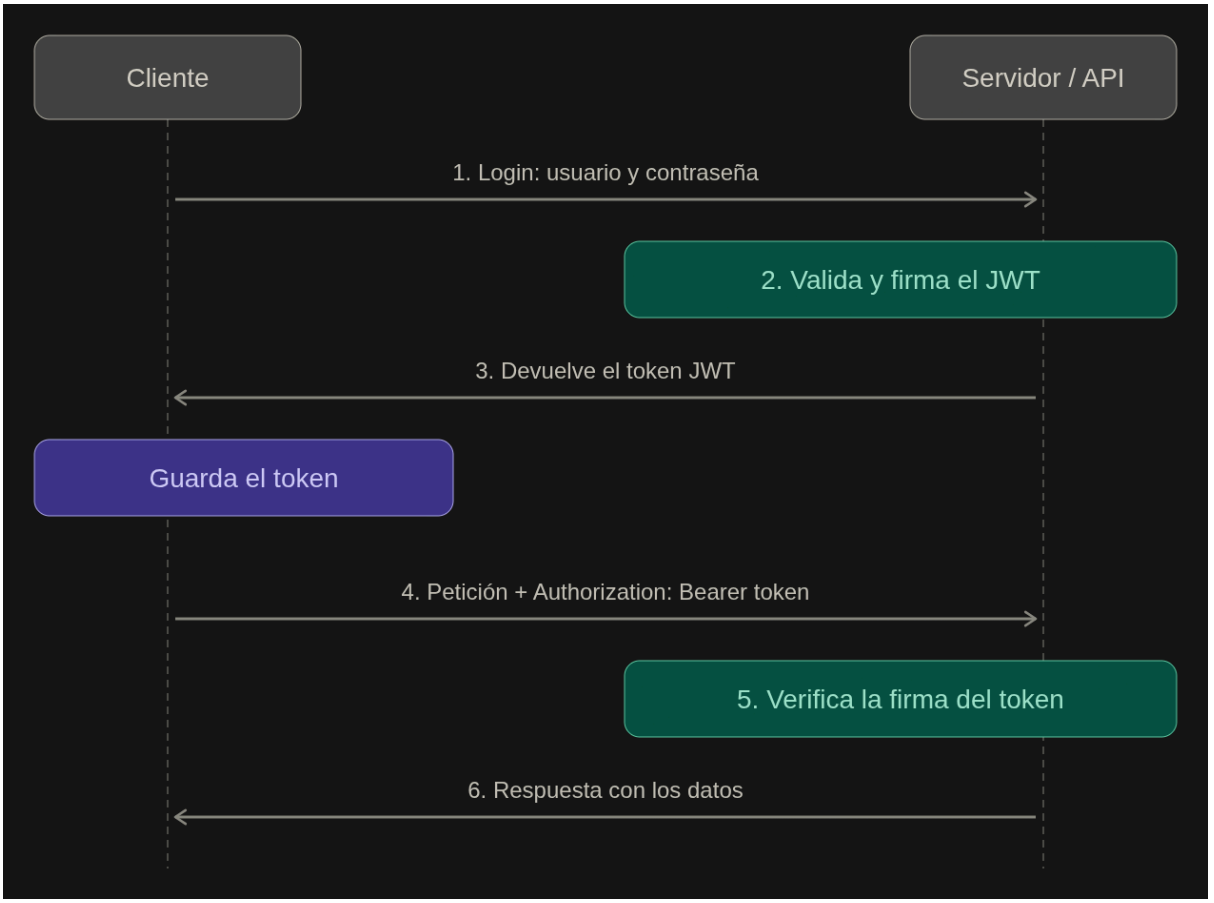
1. Significado: HTTP es HyperText Transfer Protocol (Protocolo de Transferencia de Hipertexto). HTTPS es HTTP Secure, es decir, HTTP más una capa de seguridad SSL/TLS.
2. Cifrado: en HTTP los datos viajan en texto plano. En HTTPS los datos se cifran antes de enviarse y solo el destinatario puede descifrarlos.
3. Seguridad: con HTTP, cualquiera que intercepte la conexión puede leer o modificar contraseñas, datos personales y tarjetas. Con HTTPS, la información interceptada resulta ilegible.
4. Autenticación: HTTP no verifica la identidad del sitio, por lo que es fácil suplantarlo. HTTPS usa un certificado digital emitido por una autoridad certificadora, que confirma que el sitio es quien dice ser.
5. Integridad de los datos: en HTTP el contenido puede ser alterado en el camino sin que nadie lo note. En HTTPS cualquier alteración es detectada.
6. Puerto: HTTP usa el puerto 80 por defecto. HTTPS usa el puerto 443.
7. Certificado: HTTP no necesita certificado. HTTPS requiere un certificado SSL/TLS instalado en el servidor.
8. Indicador en el navegador: con HTTP el navegador muestra la advertencia “No seguro”. Con HTTPS muestra un candado o un ícono de conexión segura.
9. Rendimiento: HTTP es ligeramente más simple porque no hay cifrado, pero la diferencia hoy es mínima. HTTPS agrega un pequeño proceso inicial (handshake), aunque con HTTP/2 y HTTP/3, que exigen HTTPS en los navegadores, suele ser igual de rápido o más.
10. Posicionamiento en buscadores (SEO): Google considera HTTPS un factor de ranking favorable. Los sitios HTTP pueden quedar penalizados.

Cómo obtener un certificado SSL gratuito

La opción más usada es **Let's Encrypt**, una autoridad certificadora gratuita y sin fines de lucro. Emite certificados de validación de dominio (DV), que son suficientes para la mayoría de los sitios. Los certificados duran poco tiempo y se renuevan automáticamente.

Json Web Token (JWT):
¿Qué es JWT (Json Web Token) y cómo funciona? Realizar una gráfica del funcionamiento.

JWT (*JSON Web Token*) es un estándar para transmitir información entre dos partes mediante un token compacto, en formato JSON y firmado digitalmente. Se usa sobre todo para autenticación y autorización en aplicaciones web y APIs.



AUTH0:
¿Qué es AUTH0 y cómo funciona? Realizar una gráfica del funcionamiento.

Auth0 es una plataforma integral de identidad como servicio (IDaaS - *Identity-as-a-Service*). Su propósito es externalizar y resolver todas las tareas de autenticación y autorización para aplicaciones web, móviles y APIs, evitando que los desarrolladores tengan que programar desde cero la seguridad, el almacenamiento de contraseñas, tokens y sesiones.

El flujo estándar de Auth0 delega la gestión de credenciales al servidor de Auth0 para que la aplicación del cliente nunca manipule directamente la contraseña del usuario:

1. **Solicitud de inicio de sesión:** El usuario hace clic en «Iniciar sesión» en la aplicación frontend o cliente.
2. **Redirección a Auth0:** La aplicación redirige al usuario a la página de login alojada por Auth0.
3. **Autenticación del usuario:** El usuario ingresa sus credenciales o se autentica mediante un proveedor social/empresarial. Auth0 valida la identidad y aplica factores de seguridad.
4. **Retorno con código de autorización:** Auth0 redirige nuevamente al usuario a la aplicación enviando un código temporal de autorización.
5. **Intercambio por tokens:** La aplicación intercambia ese código temporal con Auth0 y recibe:
    - **ID Token:** Un JWT que contiene los datos del perfil del usuario (nombre, email, avatar).
    - **Access Token:** Un JWT que certifica los permisos para comunicarse con las APIs del backend.

Consumo de la API Backend: Cada vez que la aplicación consulta datos al backend, adjunta el Access Token en la cabecera Authorization: Bearer <token>. La API valida la firma criptográfica del token con Auth0 (sin necesidad de reenviar la contraseña) y retorna los datos protegidos.

API´s Seguras:

¿Cuál es la importancia de que nuestras API's sean seguras? Detallar las buenas prácticas de programación, respecto de seguridad, que se deben tener en cuenta al momento de desarrollar una API.

La seguridad en las APIs es crítica porque son la puerta de entrada principal a la lógica de negocio, bases de datos y servicios centrales de una organización. A diferencia de una interfaz web pensada para humanos, las APIs están diseñadas para ser consumidas mediante scripts o clientes automatizados, lo que facilita ataques masivos a alta velocidad si no están protegidas.

Por qué usar APIs seguras:

- Protección de datos sensibles y confidencialidad: Las APIs manejan credenciales, información de identidad, registros de usuarios o datos financieros.
- Continuidad operativa y disponibilidad: Las APIs públicas o internas sin protección son susceptibles a saturación mediante ataques de denegación de servicio (DoS/DDoS)
- Integridad del negocio: Evita la manipulación no autorizada del estado del sistema (como alterar transacciones, saltarse pasos de validación o sobreescribir registros ajenos).

Buenas prácticas:

1. Autenticación y Autorización Robusta

   Uso de esquemas de tokens sin estado: Implementar estándares reconocidos como OAuth 2.0 u OpenID Connect, empleando firmas criptográficas mediante JWT junto con algoritmos de clave asimétrica como RS256.

   Comprobación de identidad permanente: Controlar la vigencia de firmas, receptores (aud), emisores (iss) y vencimientos (exp) dentro del token durante cada petición.

   Verificación del acceso según contexto (BFLA / BOLA): Evitar asumir permisos automáticos tras autenticar. En peticiones hacia GET /api/v1/usuarios/{id}/historial, corresponde verificar que las credenciales coincidan exactamente con el id consultado o posean atribuciones especiales.

   Restricción de privilegios: Otorgar tokens con tiempos acotados y scopes limitados, complementando el flujo mediante refresh tokens bajo resguardo.

2. Protección del Tránsito y Encriptación

   Uso imprescindible de HTTPS/TLS: Exigir protocolos TLS 1.2 o superiores en la totalidad de los puntos de acceso, rechazando o reconduciendo cualquier solicitud que circule en texto plano mediante http://.

   Implementación de encabezados de seguridad HTTP: Configurar respuestas que incorporen Strict-Transport-Security (HSTS), X-Content-Type-Options: nosniff y políticas de Content-Security-Policy (CSP) bien definidas cuando existan portales o documentación interactiva.

   Resguardo de información almacenada: Proteger claves de acceso utilizando algoritmos con sal y costo adaptable (tales como BCrypt o Argon2id), sumado al cifrado directo sobre tablas que contengan secretos o tokens de integración.

3. Sanitización y Validación Rigurosa de Entradas

   Listas de permitidos (Allow-listing): Establecer esquemas de validación estrictos con herramientas dedicadas (tales como Zod, Joi o Bean Validation) rechazando solicitudes fuera de norma.

   Prevención de asignación masiva: Utilizar objetos DTO específicos para el intercambio de datos, previniendo el mapeo directo desde el cuerpo JSON hacia los modelos de entidad.

Consultas con parámetros: Implementar sentencias preparadas o un gestor ORM seguro para mitigar vulnerabilidades de inyección SQL o NoSQL.

Comprobación de encabezados: Validar la presencia del Content-Type esperado (como application/json), rechazando payloads no identificables o maliciosos.

4. Administración de Recursos y Control de Tráfico

   Limitación de peticiones (Rate Limiting): Restringir la frecuencia de llamados por cliente, dirección IP o token para prevenir ataques DoS y fuerza bruta.

   Restricción de tamaño de carga: Definir un tope máximo para el cuerpo de la solicitud HTTP evitando el agotamiento de memoria en el servidor.

   Paginación forzosa de respuestas: Acotar la cantidad de entidades devueltas mediante parámetros de limit o cursor en consultas extensas.

5. Tratamiento de Excepciones y Datos de Salida

   Filtrado en respuestas: Mapear la información saliente hacia DTOs específicos de respuesta, omitiendo credenciales, claves internas o metadatos de la aplicación.

   Manejo global de errores: Capturar excepciones en un punto único devolviendo formatos genéricos estandarizados (como RFC 7807) sin revelar stack traces o detalles técnicos.

6. Gestión de Versiones y Ciclo de Vida

   Versionado claro de endpoints: Definir versiones en la URI (/api/v1/) para actualizar servicios sin interferir con las integraciones vigentes.

   Retiro de servicios obsoletos: Dar de baja sistemas antiguos que carezcan de los controles defensivos requeridos para evitar endpoints expuestos vulnerables.

   Publicación resguardada de esquemas: Restringir el acceso a herramientas como Swagger o consolas OpenAPI interactivas en ambientes de producción.

7. Monitoreo, Registros y Trazabilidad

   Auditoría de eventos clave: Registrar de forma segura las operaciones críticas utilizando identificadores únicos de correlación como Request-ID.

   Depuración de registros de eventos: Ocultar o sanitizar encabezados de autorización, secretos y tokens dentro de los archivos de log del sistema.